

Project Report: IoT Data Sharing Using Blockchain

Submitted by: Mahesh Krishnam (22012011/MC22BT011)
Amit Singh (220010004/CS22BT004)

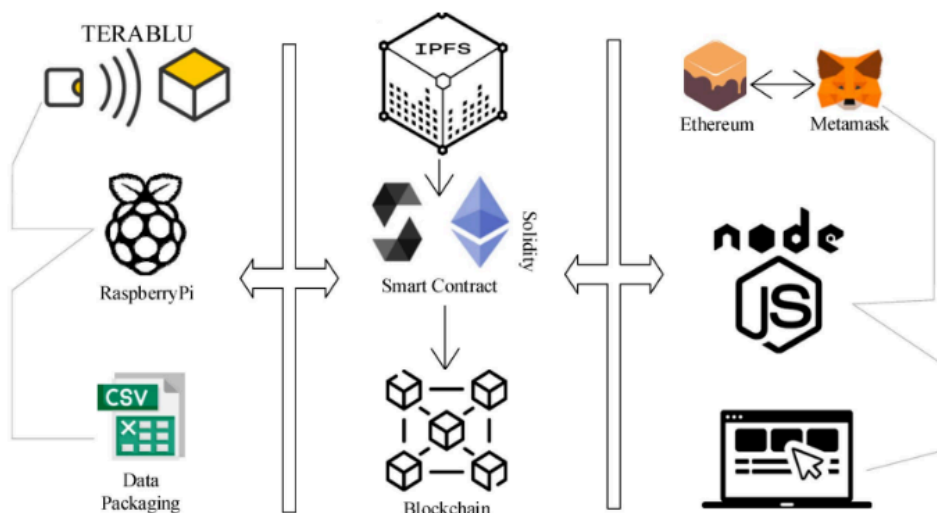
Date: April 12, 2025

Course: CS 426 : Introduction to Blockchain

Instructor: Siba Narayan Swain

1. Introduction

This project, titled "IoT Data Sharing Using Blockchain," leverages blockchain technology to create a decentralized platform for securely sharing IoT-generated data as non-fungible tokens (NFTs). The system enables IoT devices to generate data, verifiers to validate it, and list verified NFT on the marketplace, all while ensuring transparency and immutability via smart contracts deployed on the Ethereum Sepolia testnet. The project integrates with external services like Pinata for IPFS storage and OpenSea for NFT visualization, demonstrating a practical application of blockchain in IoT ecosystems.



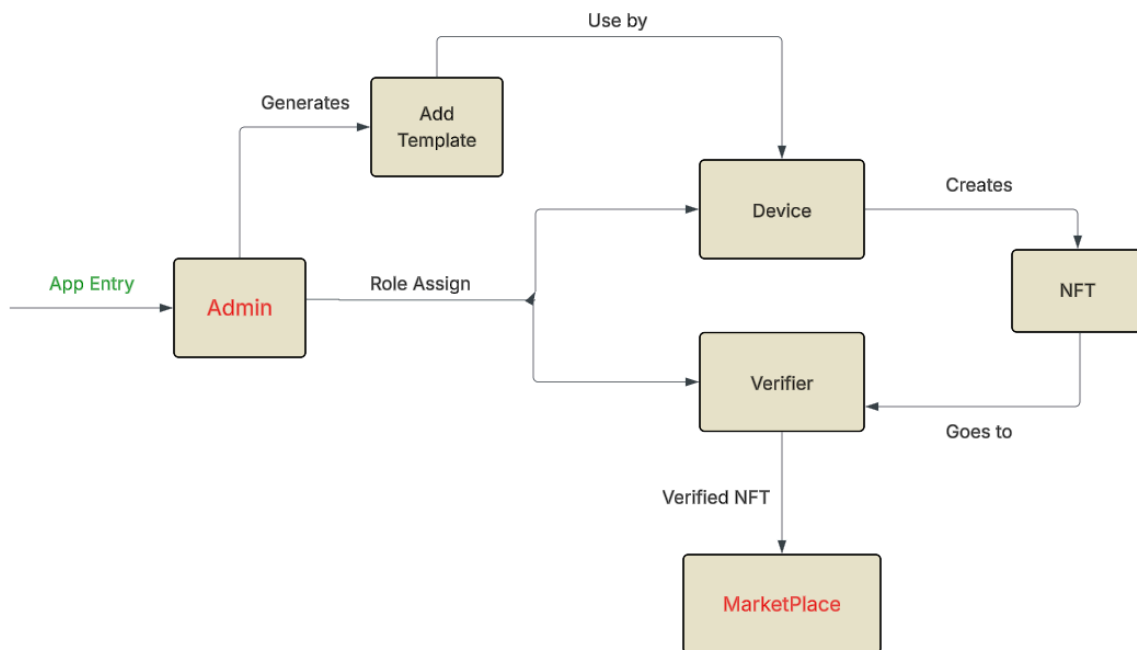
2. Objectives

- Develop a secure, decentralized system for IoT data sharing using Ethereum smart contracts.
- Enable role-based access control for admins, devices, verifiers, and users.

- Facilitate NFT creation from IoT data with metadata stored on IPFS and displayed on OpenSea.
- Implement a marketplace for trading NFTs at free of cost now (buy and sell future scope).
- Provide an intuitive frontend interface for managing roles, generating NFTs, and browsing listings.

3. System Architecture

The application architecture ,how it function :



1. Admin creating template and assigning role to device and verifier.
2. Device is creating NFTs and that NFTs goes to the verifier for verification whether uploaded NFTs are valid or not and in correct format.
3. Once verified NFTs go to MarketPlace for listing where users can get data.

3.1 Smart Contracts

- **IoTDataAccessControl**: Manages roles (Admin, Device, Verifier) with functions to grant/revoke roles and retrieve role members.
- **IoTDataFactory**: Creates data templates from admin inputs and generates NFTs using device inputs and IPFS-stored metadata.
- **IoTDataNFT**: Implements an ERC-721 token for minting NFTs, storing IoT data, and managing verification status.
- **DataVerification**: Allows verifiers to approve or reject NFT data, with status tracking.
- **Marketplace**: Enables NFT listing, buying, and cancellation.
- Reference:
 - <https://www.youtube.com/playlist?list=PLbbtODcOYIoFdQ37ydykQNO-MNGER6mtd>

- <https://www.youtube.com/watch?v=GwFQg8ROZfo>
- <https://docs.openzeppelin.com/contracts/3.x/api/token/erc721#ERC721>
- <https://www.youtube.com/watch?v=RQzuQb0dfBM&t=60s>
- <https://www.youtube.com/watch?v=eCn6mHTpuM0&t=33s>

3.2 Frontend

- Built with React, Vite, Tailwind CSS, and Wagmi/RainbowKit for blockchain interactions.
- Key components include Admin (role management), DeviceNFT (NFT generation), and a responsive UI with dim mode and enhanced styling.
- References :
 - <https://v3.tailwindcss.com/docs/guides/vite>
 - <https://wagmi.sh/>
 - <https://rainbowkit.com/docs/connect-button>
 - <https://www.youtube.com/watch?v=kzKXmzcDqwE&t=997s>

3.3 External Integrations

- **Pinata:** Stores NFT metadata on IPFS, ensuring decentralized and persistent data access.
- **OpenSea:** Displays minted NFTs on its Sepolia testnet marketplace.
- **Etherscan:** Verifies transactions on the Sepolia network.
- **MetaMask:** A browser extension that acts as a crypto wallet and gateway to blockchain apps.
- References :
 - <https://docs.pinata.cloud/frameworks/react>
 - <https://metamask.io/>
 - <https://sepolia-faucet.pk910.de/>
 - <https://sepolia.etherscan.io/>

4. Implementation Details

4.1 Smart Contract Development

- Developed using Solidity v0.8.20 with OpenZeppelin libraries for security (e.g., Ownable, ReentrancyGuard, AccessControl).
- Deployed on Sepolia using Hardhat, with ABIs and addresses integrated into the frontend via Wagmi hooks (useContractRead, useContractWrite).
- References:
 - <https://wagmi.sh/react/api/hooks>
 -
- **Key features implemented:**
 - Role management (e.g., grantDeviceRole, revokeVerifierRole).
 - NFT generation with generateDataNFT and metadata linking.

- Verification workflow with `verifyData` and status enums (PENDING, APPROVED).
- Marketplace operations (e.g., `listItem`) .

4.2 Frontend Development

- **Admin Component:** Allows admins to grant/revoke device and verifier roles, with lists of registered members. Admin is also responsible for creating templates with JSON metadata type.
- **DeviceNFT Component:** Enables devices to upload JSON data to Pinata and mint NFTs, integrating user inputs (device ID, data type, location).
- **UI Enhancements:** Implemented dim mode , gradient buttons, card-based layouts, and a sticky navbar to suggest additional features.
- **Pinata Integration:** Uploaded metadata as JSON, transformed to OpenSea standards (name, description, image, attributes) from IoT data.

5. Features and Functionalities

- **Role Management:** Admins can assign and revoke Device, Verifier, and Buyer roles.
- **NFT Generation:** Devices create NFTs from IoT data, stored on IPFS and viewable on OpenSea.
- **Data Verification:** Verifiers approve or reject NFT data, with status updates.
- **Marketplace:** NFTs approved by verifiers finally get listed in the marketplace.
- **Frontend UI:** Responsive design with dark mode, interactive buttons, and card layouts.

6. Testing and Deployment

- **Testing:** Conducted on Hardhat local node and Sepolia testnet, verifying transactions on Etherscan and NFT rendering on OpenSea.
- **Deployment:** Contracts deployed to Sepolia, frontend hosted locally with Vite, and metadata stored via Pinata.

7. Results

- Successfully implemented a functional IoT data sharing platform with blockchain security.
- NFTs are mintable and verifiable, with metadata accessible via IPFS and OpenSea.
- The marketplace supports basic trading, and the admin panel manages roles effectively.
- The frontend provides an intuitive interface, enhanced with modern styling.

8. Future Improvements

- Add batch role management and role expiration in `IoTDataAccessControl`.

- Introduce payment system for NFTs along with some fee to the platform.
- Implement auctions and price updates in the Marketplace.
- Increase the number of verifiers to verify a NFT to finally get listed on the Marketplace.
- Enhance DeviceNFT with image uploads and custom metadata editing.
- Optimize UI with loading states, tooltips, and accessibility features.
- NFTs buy and sell will require some processing fees and platform charge.

9. Conclusion

This project demonstrates the potential of blockchain for IoT data sharing, offering a secure, transparent, and decentralized solution. The integration of smart contracts, IPFS, and a polished frontend showcases technical proficiency and practical application. Challenges were addressed through debugging and iterative improvements, resulting in a robust system ready for further enhancement.

10. Acknowledgments

I extend my gratitude to my instructor, Siba Narayan Swain, for guidance, and to the internet sources and documentation for technical support during development.